

A Project Report
on
PEOPLE COUNTING SYSTEM USING DEEP LEARNING
AND
COMPUTER VISION
submitted in the partial fulfilment of the Requirements for the award of the degree.

BACHELOR OF TECHNOLOGY
IN
INFORMATION TECHNOLOGY

Submitted by
YALLAMANDA ESWAR (Regd.no:22NE1A1291)
TURLAPATI APARNA (Regd.no:23NE5A1201)
TANNIRU HARIPRIYA (Regd.no:22NE1A12B5)
MANDA JOBABU (Regd.no:22NE1A1282)

Under the Esteemed Guidance of

Mr. M. ANBARASU M. S(SS), M. Tec (CSE), (PhD)...

Associate Professor



Department of Information Technology

TIRUMALA ENGINEERING COLLEGE

An ISO9001: 2015 Certified Institution, Accredited by NAAC(A+) & NBA
(Approved by AICTE & Affiliated to JNTU, KAKINADA, Accredited by NAAC&NBA)
Jonnalagadda, Narasaraopet, GUNTUR (Dt), A.P.

2022-2026

TIRUMALA ENGINEERING COLLEGE

An ISO9001: 2015 Certified Institution, Accredited by NAAC(A+) & NBA

**(Approved by AICTE & Affiliated to JNTU, KAKINADA, Accredited by NAAC&NBA)
Jonnalagadda, Narasaraopet-522601, Guntur (Dt), A.P**

DEPARTMENT OF INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled —**PEOPLE COUNTING SYSTEM USING DEEPLARNING AND COMPUTER VISION** is the Bonafide work done by **Y. ESWAR (22NE1A1291), T. APARNA (23NE5A1201), T. HARIPRIYA (22NE1A12B5), M. JOBABU (22NE1A1282)** in partial fulfillment of the requirements for the award of “**Bachelor of Technology**” degree in the **Department of IT** from J.N.T.U. KAKINADA during the year 2022 - 2026 under our guidance and supervision and worth of acceptance of requirements of the university

Project Guide

Mr. M. ANBARASU M.S(SS), M. Tech (CSE), (Ph.D),,,

Head of the Department

Dr. J. NAGARAJU M. Tech, PhD.,

Project Coordinator

Mr. D. PAVAN KUMAR B. Tech, M. Tech, (Ph.D)

External Examiner

ACKNOWLEDGEMENT

We wish to express our thanks to various personalities who are responsible for the completion of the project. We are extremely thankful to our beloved Chairman **Sri. Bolla Brahma Naidu** and our Secretary **Sri. R. Satyanarayana**, who took a keen interest in our efforts throughout course. We owe our gratitude to our Principal **Sir Dr. Y. V. Narayana, M.E., Ph.D., FIETE**, for his kind attention and valuable guidance throughout the course

We sincerely and respectfully express gratitude to our **H.O.D Dr. J. Nagaraju M.Tech,Ph.D., Mr.D.PavanKumar.B.Tech,M.Tech,(Ph.D).**, co-ordinator of the project for extending their encouragement. Their profound knowledge and willingness have been a constant source of inspiration for me throughout the project work

We express gratitude to our guide **Mr.M.ANBARASU,M.S(SS),M.Tech(CSE),(Ph.D).**,for significant suggestions and help in every respect to accomplish the project work. His persisting encouragement, everlasting patience and keen interest in discussions have benefited us to be extent that cannot be spanned by words to our college management for providing excellent lab facilities for completion of project within our campus.

We extend our sincere thanks to all other teaching and non-teaching staff of department of **INFORMATION TECHNOLOGY** for their constant and cooperation, encouragement during our B.Tech course. I affectionately acknowledge the encouragement received from my friends and those who involved in giving valuable suggestions had clarifying out doubts which had really helped us in successfully completing my project

PROJECT ASSOCIATES

Y.ESWAR	22NE1A1291
T.APARNA	23NE5A1201
T. HARIPRIYA	22NE1A12B5
M. JOBABU	22NE1A1282

DECLARATION

We hereby declare that the project entitled **“People Counting System Using Deep Learning and Computer Vision”** submitted for the “Department of Information Technology”. This is our original work and the project has not formed the basis for the award of any degree, associateship and fellowship or any other similar titles and no part of it has been published or sent for publication at the time of submission.

By

Y.ESWAR (22NE1A1291)

T.APARNA (23NE5A1201)

T.HARIPRIYA (22NE1A12B5)

M. JOBABU (22NE1A1282)

ABSTRACT

ABSTRACT

The rapid growth of urbanization and public infrastructure has increased the need for intelligent surveillance and monitoring systems. Manual methods of counting people are time-consuming, error-prone, and unsuitable for real-time environments. This project presents a **People Counting System using Deep Learning and Computer Vision** techniques to automatically detect and count people from live webcam feeds as well as recorded video footage. The proposed system employs a **YOLO (You Only Look Once)**-based deep learning object detection model to identify human presence in video frames efficiently.

Unlike traditional approaches that rely on handcrafted features, background subtraction, or heavy tracking algorithms, this system uses an end-to-end convolutional neural network that learns robust features directly from data. A detection-first strategy with frame-wise counting is adopted, and temporal smoothing is applied to reduce fluctuations in the people count caused OUTPUT.

The system is implemented using **Python, OpenCV, and a deep learning framework**, and it requires only a single standard **RGB camera** for operation. It is designed to be lightweight, scalable, and suitable for real-time deployment on general-purpose computers. The proposed solution can be effectively used in applications such as smart surveillance, crowd monitoring, occupancy analysis, and public safety. This project demonstrates how modern deep learning techniques can be leveraged to build practical, efficient, and reliable real-time people counting systems for real-world environments.

INDEX

S. No	Title	Sub Topics	Page Number
1	Introduction	1.1 Introduction (Overview of System)	1 - 2
		1.2 Scope of the Project	3
2	Literature Survey	-	4 - 6
3	System Analysis	3.1 Existing System	7 - 8
		3.2 Proposed System	9
		3.3 System Requirements	9 - 10
		3.4 System Workflow/Analysis	10
		3.5 Algorithms Used	11
		3.6 Feasibility Study	11-12
		3.6.1 Technical Feasibility	11
		3.6.2 Operational Feasibility	11
		3.6.3 Economical Feasibility	12
		3.7 Data Preprocessing	12–13
4	System Design	4.1 Class Diagram	14-15
		4.2 Use Case Diagram	16-17
		4.3 Activity Diagram	18-19
		4.4 Component Diagram	20
		4.5 Sequence Diagram	21-22
5	Implementation	-	23-34
6	Result Analysis	-	35-38
7	Output Screenshots	-	41-47
8	Conclusion	-	48-49
9	Future Enhancement	-	50-52
10	Bibliography	-	53-54

CHAPTER–1

INTRODUCTION

INTRODUCTION

The rapid growth of urbanization and public infrastructure has increased the need for intelligent surveillance and monitoring systems. Manual methods of counting people are time-consuming, error-prone, and unsuitable for real-time environments. This project presents a **People Counting System using Deep Learning and Computer Vision** techniques to automatically detect and count people from live webcam feeds as well as recorded video footage.

The proposed system employs a **YOLO (You Only Look Once)**-based deep learning object detection model to identify human presence in video frames efficiently.

Unlike traditional approaches that rely on handcrafted features, background subtraction heavy tracking algorithms, this system uses an end-to-end convolutional neural network learns robust features directly from data. A detection-first strategy with frame-wise count is adopted, and temporal smoothing is applied to reduce fluctuations in the people count by occlusion or partial visibility.

The system is implemented using **Python, OpenCV, and a deep learning framework**, and it requires only a single standard **RGB camera** for operation. It is designed to be lightweight, scalable, and suitable for real-time deployment on general-purpose computers. The proposed solution can be effectively used in applications such as smart surveillance, crowd monitoring, occupancy analysis, and public safety. This project demonstrates how modern deep learning techniques can be leveraged to build practical, efficient, and reliable real-time people counting systems for real-world environments.

1.1 Introduction (Sub-topic)

This section explains the basic idea of the system. The People Counting System is designed to automatically detect and count the number of people present in a particular area using an RGB camera. The system focuses on delivering accurate and reliable results in real-time environments.

is implemented using Python, OpenCV, and a deep learning framework, and requires only A single standard RGB camera for operation. The system is lightweight, scalable, and suitable for deployment on general-purpose computers.

The proposed solution can be effectively used in applications such as smart surveillance, crowd monitoring, occupancy analysis, and public safety. This project demonstrates how modern deep learning techniques can be utilized to build practical and efficient real-time people counting systems for real-world environments

1.1 Scope of the Project exit monitoring.

Enhances workplace security. The scope of the People Counting System is wide and continuously growing. It can be applied in multiple domains such as

Shopping Malls: Helps track customer footfall. Supports business analytics and staffing decisions.

Airports and Railway Stations: Monitors passenger movement. Improves crowd control and safety management.

Educational Institutions: Tracks classroom attendance automatically. Monitors student movement in campus areas.

Offices and Corporate Buildings: Controls entry and Prevents overcrowding in sensitive areas.

Hospitals: Manages visitor flow.

Smart Cities: Supports intelligent traffic and crowd -monitoring systems. Helps urban planning and infrastructure management.

Event Management: Tracks crowd size during concerts, festivals, and public gatherings. Ensures safety compliance.

Chapter-2

LITERATURE SURVEY

Table:2.1Literaturesurvey

S. No	Author	Title	Year	Publication	Description
1	Redmon et al.	You Only Look Once: Unified, Real-Time Object Detection	2016	IEEECVPR	Introduced the YOLO object detection framework, enabling fast real time detection by processing images in a single pass. This work laid the foundation for real-time people detection systems.
2	Redom & Farhadi	YOLO9000: Better, Faster, Stronger	2017	IEEECVPR	Improved YOLO by enhancing detection accuracy and speed. Demonstrated the ability to detect multiple object classes efficiently, making it suitable for surveillance applications.
3	Wojke et al.	YOLOv4: Optimal Speed	2021	ar X iv	Improved YOLO architecture with better performance.
4	Zhang et al.	YOLOv8 Model Documentation	2023	Ultralytics	Latest YOLO version with improved accuracy and speed.
5	Zhang et al.	Real- Time People Counting Using YOLO and Deep Sort	2024	MDPI /IEEE	Hybrid models combing detection and tracking

Detailed Explanation of Literature Survey

People counting systems are important for monitoring crowd movement in places such as shopping malls, railway stations, educational institutions, and offices. Traditional manual counting methods are time-consuming and less accurate. Therefore, automated systems using computer vision and deep learning techniques are developed to improve efficiency and reliability.

2. Traditional People Counting Methods

Earlier systems used sensors such as infrared and pressure sensors to count people. These systems worked only at entry and exit points and were not suitable for crowded environments. Their accuracy was also limited in real-time applications.

3. Image Processing- Based Approaches

Researchers later introduced image-processing techniques such as Haar Cascade classifiers, Histogram of Oriented Gradients (HOG), and background subtraction methods. These methods improved detection performance but were affected by lighting conditions, shadows, and overlapping objects.

4. Deep Learning-Based Detection Techniques

Modern systems use deep learning models such as YOLO, Faster R-CNN, and Mobile Net-SSD for detecting people in video streams. These models provide higher accuracy and faster processing, making them suitable for real-time surveillance applications.

Tracking-Based People Counting Systems

Tracking algorithms such as DeepSORT and centroid tracking help maintain identity across frames and reduce duplicate counting errors. These techniques improve the overall accuracy of people counting systems.

5. Advanced Monitoring Systems

Recent systems integrate cloud storage, dashboards, and heatmap visualization for better monitoring and analysis. These features support real-time data access and help in effective crowd management.

6. Conclusion of Literature Survey

The literature survey shows that deep learning-based people counting systems provide better accuracy and performance compared to traditional sensor-based and image processing methods.

Chapter-3

SYSTEM ANALYSIS

System Analysis is an important phase in the project where the current system is studied, problems are identified, and a better solution is proposed. It helps in understanding how the system should function and what improvements are needed.

3.1 Existing System

The existing people counting systems mainly rely on **manual observation, sensor-based techniques, and traditional image-processing methods** to monitor the number of people in a particular area.

Working of Existing System:

1. The system captures data using surveillance cameras or sensors such as infrared and pressure sensors placed at entry and exit points.
2. Traditional image-processing techniques like Haar Cascade classifiers, Histogram of Oriented Gradients (HOG), or sensor signals are used to detect the presence of people.
3. The system tracks whether a person is entering or exiting the monitored area based on motion detection or sensor triggering.
4. The system increases or decreases the count depending on the detected movement direction at entry and exit points.

The total number of people is displayed on a monitoring screen or stored for later reference by administrators.

Limitations in Detail:

1. **Dependence on manual monitoring**, which increases workload and leads to human errors.
 2. **Limited accuracy in crowded environments**, especially when multiple people move together.
 3. **Restricted coverage area**, as sensor-based systems work only at entry and exit points.
- e nsitivity to lighting conditions, shadows, and background variations.
3. Difficulty handling overlapping people (occlusion problem).

4. Lack of real-time analytics and remote monitoring support.
5. Limited scalability for large-area and multi-camera surveillance systems.

3.2 Proposed System (Detailed)

The proposed system is an **People counting system using deep learning and CV**

That enhances traditional TTS systems by adding intelligence and structure.

Working of Proposed System:

The system captures video frames continuously from surveillance cameras

1. A deep learning-based object detection model such as YOLO detects people accurately from the captured video frames.
2. The detected persons are tracked frame by frame using tracking algorithms to maintain identity and avoid duplicate counting.
3. The system counts people by analyzing their movement across a predefined virtual line to determine entry and exit.
4. The counted data is displayed on a live dashboard and stored in a database for future monitoring and analysis.

Detail:

Tracking algorithms help count multiple people even when they move together or overlap. The system can monitor large spaces such as malls, railway stations, campuses, and offices. Helps control overcrowding and enhances surveillance in public places. Reduce human effort.

3.3 System Requirements (Detailed)

System requirements define the necessary hardware and software resources required to run the Real-Time People Counting System efficiently.

Hardware Requirements

Processor: Minimum Intel i3 or equivalent processor required for smooth execution of video processing and detection algorithms

- **RAM:** At least 4GB RAM recommended for handling real-time frame processing and object detection
- **Storage:** Required for storing datasets, trained models, and recorded counting data
- **Camera:** Webcam or CCTV camera for capturing real-time video input
- **Display Device:** Monitor required to visualize detection results and dashboard output.

Software Requirements:

Operating System: Compatible with Windows, Linux, or macOS

- **Programming Language:** Python (preferred for computer vision and deep learning tasks)

Libraries:

OpenCV – For video processing and frame extraction NumPy – For numerical computations

YOLO/TensorFlow/Py Torch – For person detection model Pandas – For storing and managing counting data

Development Tools: VS Code, PyCharm, Jupyter Notebook, Notepad

3.4 Analysis

This section describes the internal working process of the people counting system step by step.

Step-by-Step Workflow

1 Input Stage:

The system captures real-time video using webcam or CCTV camera.

2 Frame Extraction Stage:

Video stream is converted into multiple frames o Frames are processed continuously for analysis.

3 Person Detection Stage:

Deep learning model detects human presence in each frame Bounding boxes are created around detected persons.

4 Object Tracking Stage:

Tracking algorithm follows detected persons across frames o Prevents duplicate counting of the same person

5 Counting Stage:

People crossing a predefined virtual line are counted o Entry and exit counts are updated automatically

6 Output Stage:

Total people count is displayed on screen Data is stored for monitoring and analysis.

3.5 Algorithms:

The system uses multiple algorithms to ensure efficient detection and counting performance

1. Object Detection Algorithm (YOLO)

Detects people from videoframes in real time

- Provides high-speed and high-accuracy detection

2. Frame Processing Algorithm

Converts video stream into frames

- Processes frames sequentially for analysis

3. Object Tracking Algorithm

Tracks movement of detected persons across frames

- Prevents duplicate counting errors

4. Line Crossing Algorithm

Detects when a person crosses a predefined counting line

- Updates entry and exit count automatically

5. Deep Learning Model Optimization

Improves detection accuracy

- Reduces false detection during real-time monitoring

3.6 Feasibility Study:

Feasibility study evaluates whether the proposed system can be implemented successfully.

3.6.1 Technical Feasibility

Uses widely available technologies such as Python, OpenCV, and deep learning models

Easy integration with existing surveillance cameras Requires minimal hardware configuration

Supports future upgrades like multi-camera integration

3.6.1 Operational Feasibility

User-friendly monitoring interface

Requires minimal technical knowledge to operate

Useful in educational institutions, malls, offices, and public places Improves surveillance efficiency and safety monitoring.

3.6.2 Economic Feasibility

Low development cost using open-source tools No expensive hardware required

Maintenance cost is minimal

Suitable for small-scale and large-scale deployment

Data Preprocessing

Data preprocessing improves the performance of the detection system before applying deep learning models.

Detailed Steps Frame Extraction

Converts video stream into individual frames for processing

Image Resizing

Adjusts frame size for faster detection processing

Noise Reduction

Removes unwanted disturbances from frames

Object Detection Preparation

Enhances image clarity for accurate detection

Bounding Box Generation

Identifies detected persons using rectangular boxes

Frame Normalization

Improves image consistency for better model performance

Importance in Detail Improves detection accuracy

- Reduces false positives during counting
- Enhances tracking performance
- Ensures smooth real-time monitoring
- Makes the system reliable in different environments

Chapter-4

DESIGN

4.1 Class Diagram

The Class Diagram represents the structure of the Real-Time People Counting System by showing system classes, their attributes, methods, and relationships. It explains how different modules interact to perform detection and counting operations.

Classes in the System

1. Video Stream Handler Methods:

Capture Video ()– Captures video input from webcam or CCTV Extract Frames ()– Converts video into frames for processing **Role:**

Handles video input and prepares frames for further analysis.

2. Person Detector

Methods:

Load Model ()– Loads trained deep learning model Detect Persons ()– Detects people from video frames

Role:

Responsible for detecting human presence using deep learning algorithms such as YOLO.

Object Tracker Methods:

Initialize Tracker ()–Initializes tracking process

Track Movement ()–Tracks detected persons across frames

Role:

Tracks individuals to avoid duplicate counting and maintain identity consistency.

3. People Counter Methods:

Count Entry ()–Counts people entering monitored area

Count Exit ()–Counts people exiting monitored area Update Total Count ()–Updates overall people count

Role:

Core component responsible for counting people based on movement direction.

Relationships Between Classes:

Video Stream Handler → Person Detector (Frames are sent for detection)

Person Detector → Object Tracker (Detected persons are tracked)

Object Tracker → People Counter (Tracked movement used for counting)

People Counter → Database Manager (Count stored for future use)

Summary:

The Class Diagram shows a **modular architecture**, where each class has a specific responsibility, making the system easy to maintain and scalable.

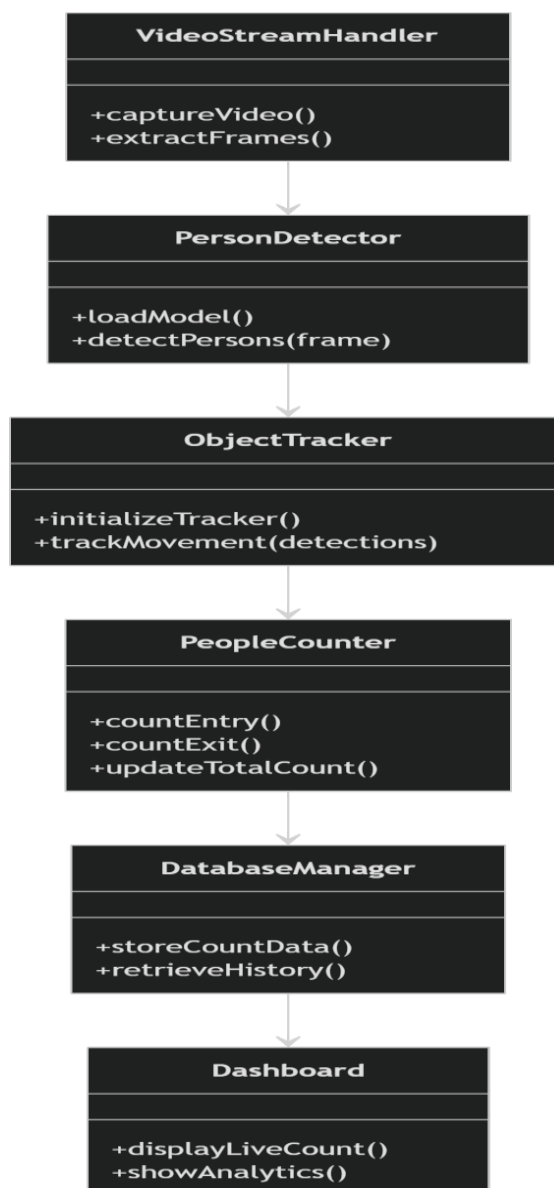


Figure:4.1.1ClassDiagram

4.2 Use Case Diagram:

The **Use Case Diagram** represents the interaction between the user and the Real-Time People Counting System. It shows how the administrator monitors and controls the system while the system performs automatic detection and counting operations.

Actor

- **Admin/User**—The person who interacts with the system to monitor people count through the dashboard interface

Use Cases

1. Start Camera Monitoring

User starts the surveillance system to capture live video from the camera

2. Detect People

System detects people automatically from video frames using deep learning models

3. Track Movement

System tracks the movement of detected persons across frames to maintain identity

4. Count People

System updates entry and exit count when a person crosses the predefined counting line

5. View Live Count

User monitors real-time people count through the dashboard interface

6. Store Count Data

System stores people counting information for future analysis and reporting

Relationships

- The **Admin/User interacts** with monitoring and viewing system results
- **“Count People” includes:**
 - 1 Detect People
 - 2 Track Movement
 - 3 Update Count Data

Summary:

This diagram clearly explains how the user interacts with the system dashboard while the system automatically performs video capture, detection, tracking, counting, and data storage operations. It represents the overall functionality provided by the Real-Time People Counting System.

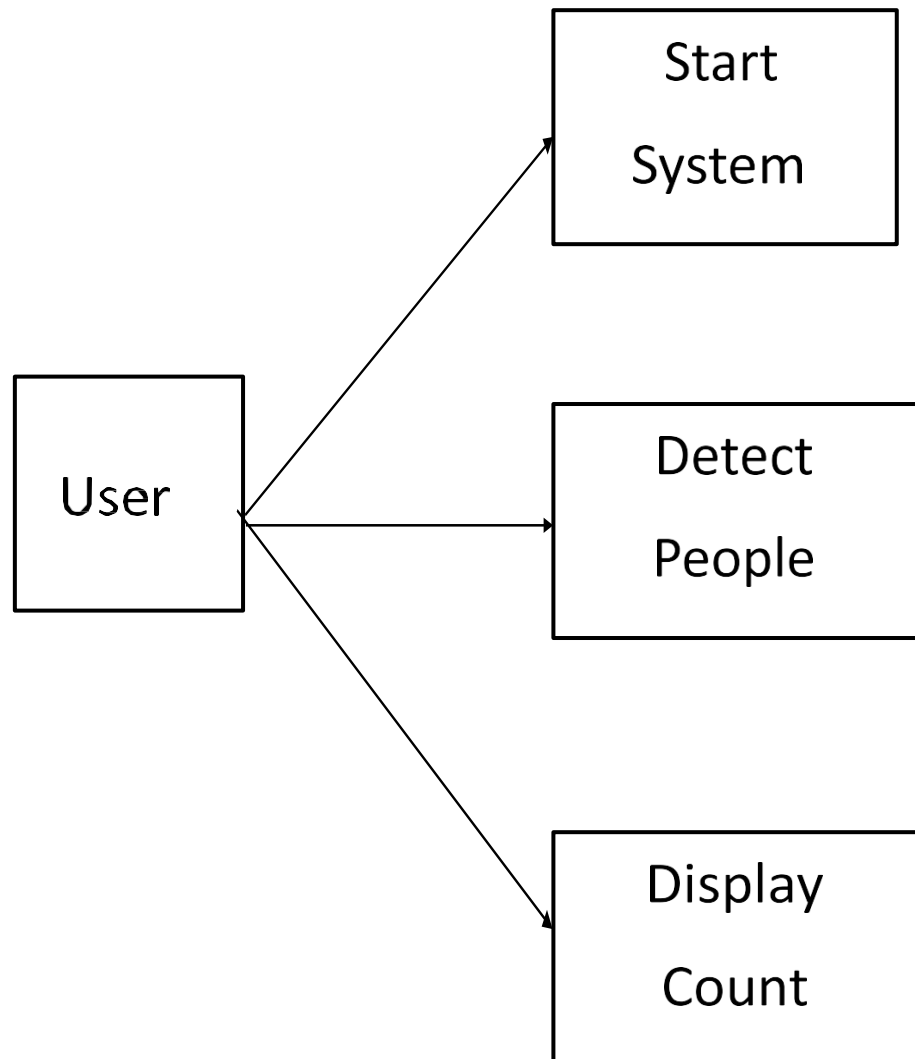


Figure: 4.2.1Usecasediagram

4.3 Activity Diagram

The **Activity Diagram** shows the work flow of the system step-by-step from video capture to people counting and display of results.

Flow of Activities

Start

Start Camera Monitoring

System captures live video from webcam or CCTV camera

Extract Frames

Video stream is converted into frames for processing

Detect People

Deep learning model detects people from each frame

Track Movement

System tracks detected persons across frames

Decision: Crossed Counting Line?

System checks whether the detected person crossed the predefined virtual counting line

Possible Flows

If Yes

Update People Count Store Count Data Display Updated Count Continue Monitoring

If No
Continue Monitoring Return to Detection Stage

Key Features

- Shows decision-making using a condition (line crossing detection)
- Clearly represents continuous monitoring workflow
- Explains how entry and exit counting is performed automatically

Summary

This diagram explains how the system operates internally from start to end by capturing video, detecting people, tracking movement, counting entries/exits, and displaying results on

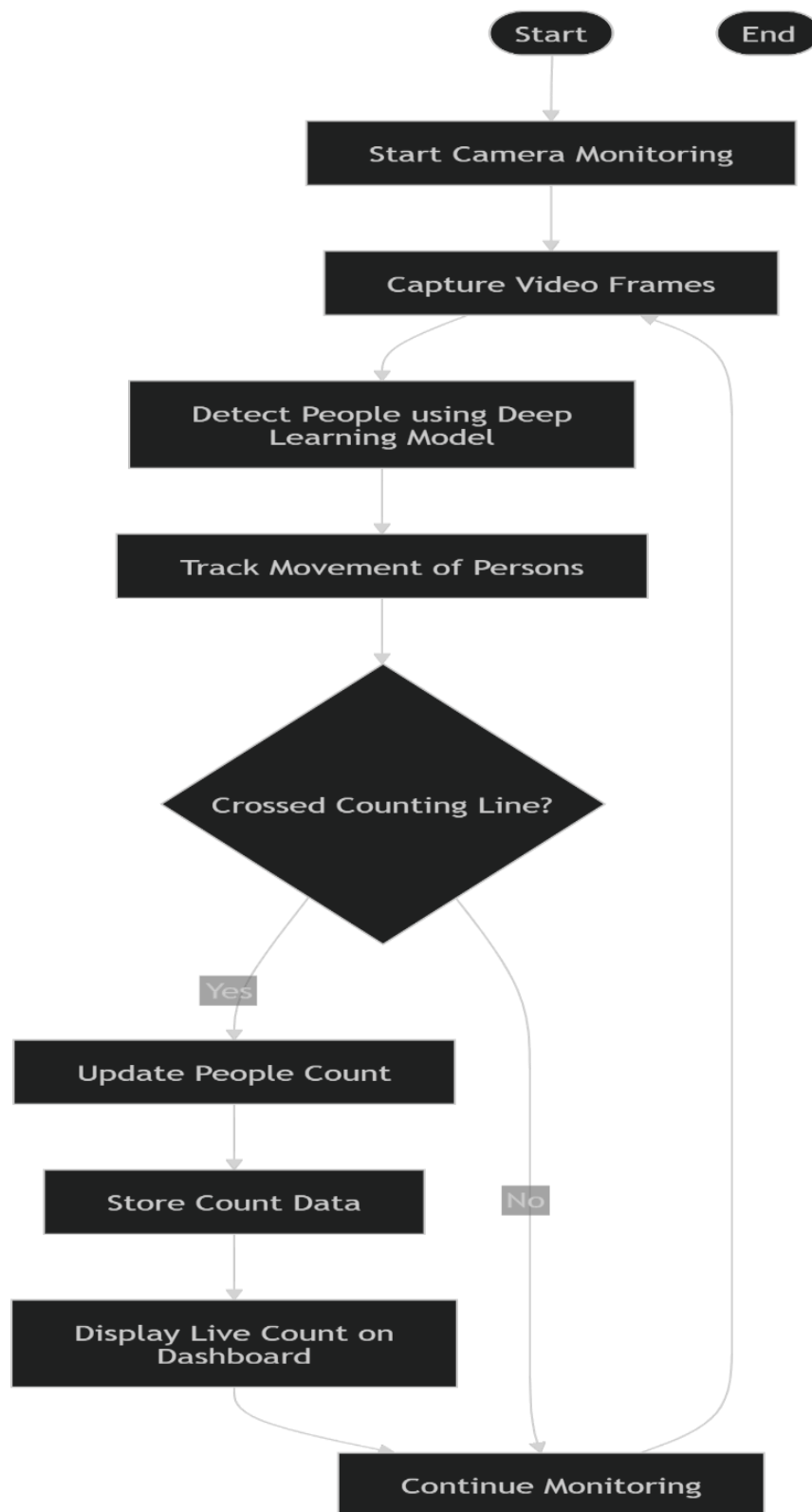


Figure:4.3.1Activitydiagram

4.4 Component Diagram

The Component Diagram represents the physical architecture of the system and shows how different components interact to perform people detection, tracking, counting, and monitoring operations.

Components in the System

1. User Interface

- Displays real-time people count
- Allows user to monitor system output through dashboard
-

2. Video Capture Module

- Captures live video input from webcam or CCTV camera
- Sends video frames to detection module for processing

3. Person Detection Module

- Detects people from captured video frames using deep learning models such as YOLO
- Sends detected person data to tracking module

4. Object Tracking Module

- Tracks movement of detected persons across frames
- Helps prevent duplicate counting errors

5. People Counting Module

- Counts entry and exit of people based on movement direction
- Updates total number of people present

6. Storage System

- Stores counted data for future reference

- Maintains history records for monitoring and analysis
-

Interactions

- User Interface→ Video Capture Module
- Video Capture Module→ Person Detection Module
- Person Detection Module→ Object Tracking Module
- Object Tracking Module→ People Counting Module
- People Counting Module→ Storage System

Summary

This diagram shows how different modules of the Real-Time People Counting System are connected and how video data flows from capture to detection, tracking, counting, and storage. It represents the modular architecture of the system, which improves efficiency, scalability, and maintainability.

4.5 Sequence Diagram

The **Sequence Diagram** represents how different components of the Real-Time People Counting System interact with each other over time to perform people detection and counting operations.

It shows the step-by-step communication between the User, Camera Module, Detection Module, Tracking Module, Counting Module, and Display Module. Flow of Interaction

1. Start Monitoring

The process begins when the user starts the system for monitoring people movement.

2. Capture Video Frames

The **Camera Module** continuously captures frames from:

- Recorded video file
- Webcam

These frames are forwarded to the detection module.

3. Detect People using YOLOv5

The **Detection Module** processes each captured frame and:

- Identifies objects in the frame
- Filters only the person class

Draws bounding boxes around detected persons This step uses the YOLOv5 deep learning model.

4. Send Detected Persons to Counting Module

After **detection**:

- Bounding box coordinates are extracted
- Person objects are forwarded to counting module

5. Count Number of People

The **Counting Module**:

- Counts detected bounding boxes
- Removes false detections
- Updates total people count

6. Send Count Data to Display Module

The updated count value is sent to the display module for visualization.

7. Display Live People Count

The **Display Module:**

- Draws bounding boxes
- Displays live people count on screen
- Shows processed video output window

Key Features of Sequence Diagram

- Shows interaction between system modules step-by-step
- Explains execution order of operations
- Represents real-time processing workflow
- Helps understand communication between components clearly

Summary

The Sequence Diagram explains how the Real-Time People Counting System processes video input step-by-step from capturing frames to detecting people and displaying the live count on the screen. It illustrates the interaction between system components and ensures smooth real-time monitoring functionality

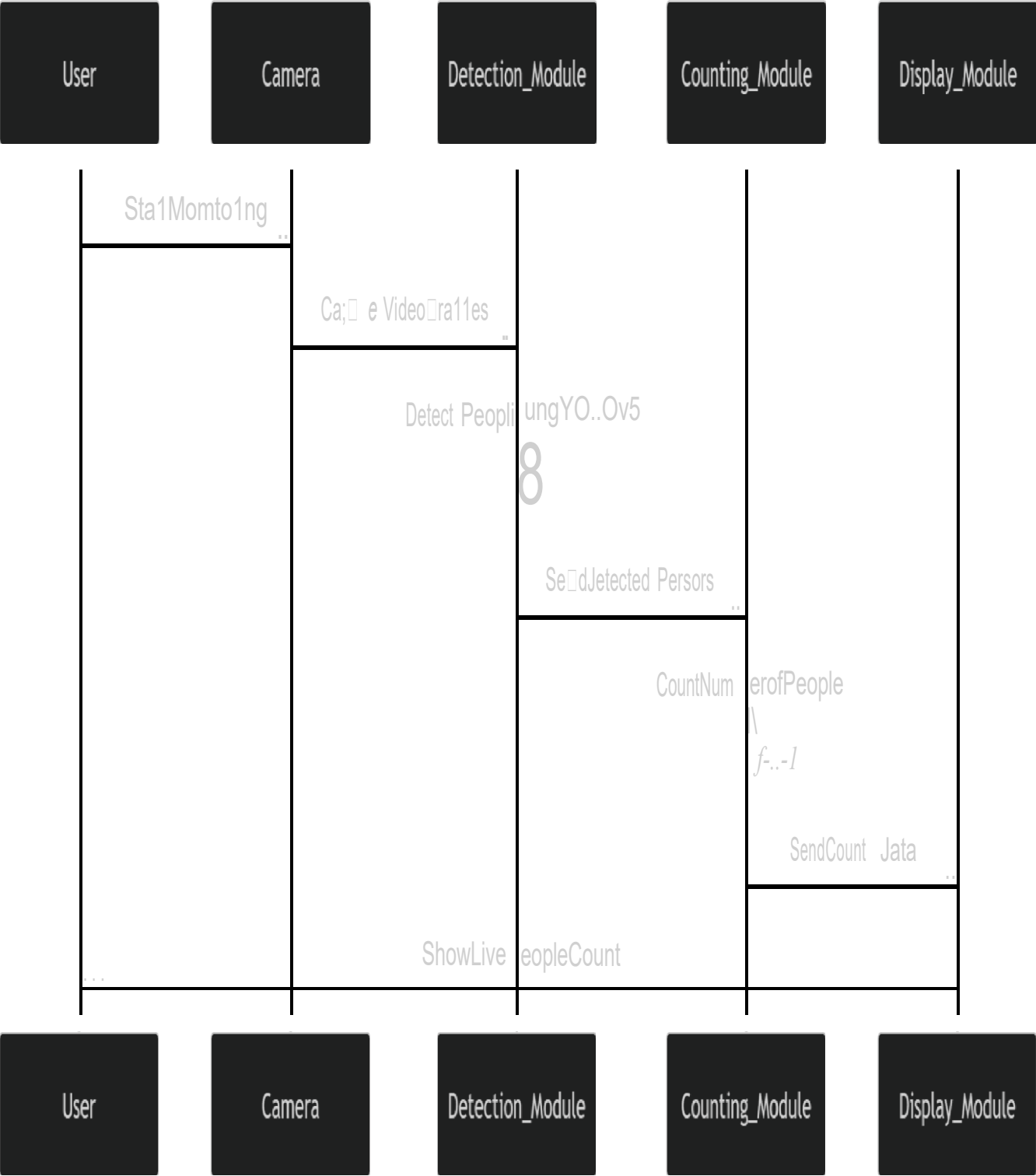


Figure:4.5.1Sequencediagram

Chapter-5

IMPLEMENTATION

The **Implementation phase** is where the designed Real-Time People Counting System is developed using programming languages, deep learning frameworks, and computer vision techniques. In this phase, different modules such as video capture, person detection, tracking, and counting are integrated to build a complete working system.

4.2 Overview of Implementation

The system is implemented using the **Python programming language** because of its strong support for **Computer Vision and Deep Learning libraries**. The implementation follows a **modular approach**, where each functionality is designed as an independent module to improve flexibility, scalability, and maintenance.

4.3 Modules in Implementation

1. Video Input Module

- Captures live video from webcam or CCTV camera
- Provides continuous frame input to the processing module

Tools Used:

- OpenCV(cv2library)

2. Frame Preprocessing Module

- Prepares captured frames before detection

Functions:

- Resize frames
- Remove noise
- Improve frame quality
- Convert frame formats if required

Libraries Used:

- OpenCV
- NumPy

3. Person Detection Module

- Detects people from video frames using deep learning models

Functions:

- Identify human objects in frames
- Draw bounding boxes around detected persons
- Send detected objects to tracking module

Libraries Used:

- YOLO (You Only Look Once)
- OpenCV
- TensorFlow/Py Torch

4. Object Tracking Module

- Tracks detected persons across multiple frames

Functions:

- Assign unique ID to each detected person
- Track movement direction
- Prevent duplicate counting

Libraries Used

- Centroid Tracking Algorithm
- Deep SORT (optional)

5. People Counting Module

- Counts entry and exit of people based on movement

Functions:

- Detect line crossing events
- Update total people count
- Maintain entry/exit statistics

6. Output Display Module

- Displays processed output to the user

Features:

- Show live people count on screen

- Display bounding boxes
- Show tracking IDs

7. Storage Module

- Stores counting results for analysis and reporting

Functions:

- Save count data in files or database
- Maintain history logs

4.4 Step-by-Step Implementation Process

Camera captures live video input

Video frames are sent to preprocessing module Frames are passed to person detection module Detected persons are tracked across frames

System checks whether a person crosses counting line People count is updated automatically Output is displayed on the monitoring screen Count data is stored for future reference

4.5 Tools and Technologies Used

Category	Tools/Technologies
Programming Language	Python
Computer Vision Library	OpenCV
Deep Learning Model	YOLO
Tracking Algorithm	Centroid Tracking/Deep SORT
IDE	VS Code/PyCharm/Command Prompt
Operating System	Windows/Linux/macOS

Code Structure (Basic Overview)

The project is organized into multiple modules: video_capture.py → Captures video frames

preprocessing.py → Frame preprocessing

person_detection.py → Detects people using YOLO

tracking.py → Tracks detected persons counting.py → Counts entry and exit display.py →

Displays output on screen main.py → Integrates all modules

Challenges Faced During Implementation Detecting people accurately in crowded environments
Avoiding duplicate counting errors

Maintaining tracking accuracy across frames Handling lighting and camera angle variations

Processing video frames in real time efficiently

4.6 Testing During Implementation

Unit testing for each module

Integration testing for complete system workflow Accuracy testing for people detection

Performance testing for real-time processing Output verification using sample video datasets

SOURCE CODE

DESCRIPTION OF YOLO MODEL:

Arguments received: ['yolo']. Ultra lytics 'yolo' commands use the following syntax: yolo TASK MODE ARGS

Where TASK (optional) is one of ['classify', 'detect', 'pose', 'segment', 'obb'] MODE (required) is one of ['val', 'benchmark', 'export', 'track', 'train', 'predict'] ARGS(optional)areanynumberofcustom'arg=value'pairslike'imgsz=320'that See all ARGS at <https://docs.ultralytics.com/usage/cfg> or with 'yolo cfg'

1. Trainadetectionmodelfor10epochswithaninitiallearning_rateof0.01yolo train

data=coco8.yaml model=yolo11n.pt epochs=10 lr=0.01

Predict a You Tube video using a pretrained segmentation model at imagesize320: yolo predict model=yolo11n-seg.pt source='https://youtu.be/LNwODJXcvt4'imgsz=320

2. Validateapretraineddetectionmodelatbatch-size1andimagesize640: yolo val

model=yolo11n.pt data=coco8.yaml batch=1 imgsz=640

3. ExportaYOLO11classificationmodeltoONNXformatatimagesize224by128 (no TASK required)

yoloexportmodel=yolo11n-cls.ptformat=onnximgsz=224,128

4. Ultra lytics solutions usage

yolosolutionscountoranyof['crop','blur','workout','heatmap','isegment','visioneye','speed','queue','analytics','inference','trackzone'] source="path/to/video.mp4"

5. Runspecialcommands:

yolo helpyolo checksyolo versionyolo settingsolocopy-cfg yolo cfg

yolo solutions help

1. Purpose

- The message is a usage guide for the YOLO (You Only Look Once) command-line interface (CLI).
- Itappearswhen'yolo'isrunwithoutcorrectargumentsorinthewrongformat.

2. Command Syntax

- YOLO commands follow the structure: `yolo TASK MODE ARGS` where:

TASK (optional) – Type of computer vision task: ['classify', 'detect', 'pose', 'segment', 'obb'] **MODE** (required)–Action to perform: ['train', 'predict', 'val', 'benchmark', 'export', 'track']

ARGS (optional)–Additional parameters in 'key=value' format to override defaults **Examples:**
`imgsz=640, epochs=10, lr0=0.01`

3. Examples Provided

train a detection model: `yolo train data=coco8.yaml model=yolo11n.pt epochs=10 lr0=0.01`

Predict on a YouTube video using a segmentation model: `yolo predict model=yolo11n-seg.pt source='https://youtu.be/LNwODJXcvt4' imgsz=320`

-Validate a pretrained detection model:

`yolo val model=yolo11n.pt data=coco8.yaml batch=1 img sz=640`

-Export a YOLO model to ONNX format: `yolo export model=yolo11n-cls.pt format= onnx img sz=224,128`

- Use built-in Ultra lytics “solutions”:

`yolo solutions count source="path/to/video.mp4"`

-Special utility commands: `yolo help, yolo checks, yolo version, yolo settings, yolo cfg`

4. Why This Appears

- The CLI shows this guide if you run 'yolo' with missing or incorrect TASK/MODE/ARGS.
- It is meant to guide users on how to properly use YOLO commands.

5. Key Points

- TASK specifies the vision problem type.
- MODE specifies what YOLO should do (train, predict, validate, etc.).
- ARGS are optional custom parameters to fine-tune the behavior.
- YOLOCLI provides ready-to-use solutions and utility commands for convenience

SOURCE CODE:

```
import cv2
import torch

model = torch.hub.load('ultralytics/yolov5', 'yolov5s')
model.conf = 0.5
PERSON_CLASS = 0

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break
    frame = cv2.resize(frame, (640, 480))
    results = model(frame)
    detections = results.xyxy[0]
    boxes = []
    for *box, conf, cls in detections:
        if int(cls) == PERSON_CLASS and conf > 0.5:
            x1, y1, x2, y2 = map(int, box)
            width = x2 - x1
            height = y2 - y1
            # remove false detections if width < 100 or height < 100:
            continue
            boxes.append([x1, y1, x2, y2])
    count = len(boxes)
    for (x1, y1, x2, y2) in boxes:
        cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
    cv2.putText(frame, f'People Count: {count}',
                (20, 40), cv2.FONT_HERSHEY_SIMPLEX, 1,
                (0, 0, 255), 2)
    cv2.imshow("People Counter", frame)
    if cv2.waitKey(1) == 27:
        break
    cap.release()
    cv2.destroyAllWindows()
```

Overall Explanation–YOLOv5 Real-Time People Counting (Webcam)

1. Import Libraries

- cv2: OpenCV for video capture, frame processing, drawing, and display
- torch: PyTorch for loading and running the YOLOv5 model

2. Load YOLOv5 Model

- Load a pre-trained YOLOv5 small model ('yolov5s')
- Set confidence threshold (0.5) to filter low-confidence detections
- PERSON_CLASS=0(YOLOv5classID for people)

3. Initialize Webcam

- cv2.VideoCapture (0) opens the default camera
- Checks if frames can be read successfully

4. Process Video Frames in Loop

- Capture each frame from the webcam
- Resize frame to 640 x 480 for consistent detection
- Pass frame to YOLOv5 for object detection

5. Filter Detections

- Keep only detections with class ID0 (person)
- Only consider boxes with confidence>0.5
- Ignore very small boxes (width or height<100) to reduce false positives

6. Count and Draw

- Count the number of valid people detected
- Draw green bounding boxes around each detected person
- Overlay total people count on the frame in red text

7. Display Video

- Show the processed frame in a window titled "People Counter"

8. Exit Condition

- Break the loop if the ESC key (ASCII27) is pressed

9. Release Resources

- Release the webcam using cap. release ()
- Close all Open CV Windows using cv2. destroy All Windows ()

Key Features:

- Real-time people detection and counting
- Visual feedback with bounding boxes and count
- Reduces false positives using confidence and size filtering
- Runs continuously until manually stopped

SOURCE CODE:

```
import cv2 import torch
model= torch. hub. load ('ultra lytics/yolov5','yolov5n') model. conf=0.3
PERSON_CLASS = 0
cap=cv2.VideoCapture("smart.mp4") if
not cap. is Opened ():
print ("Error: Could not open video.") exit ()
frame_skip=2
frame_count=0
while True:
ret, frame = cap. read () if not
ret:
break frame_count +=1
if frame_count% frame_skip!=0: continue
frame=cv2.resize(frame, (640,360)) results
= model(frame) detections=results.
xyxy[0]
count = 0
for*box, conf, c ls in detections:
if int (c ls)==PERSON_CLASS: count += 1
```

```

    x1, y1, x2, y2 = map(int, box) cv2.rectangle(frame, (x1, y1), (x2,
y2), (0,255,0),2) cv2.putText (frame, f" People Count: {count}",
(20,40), cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255),2)
cv2.imshow("People Counter", frame) if
cv2.waitKey(1) == 27:
    break cap.
release ()
cv2.destroyAllWindows()

```

Overall Explanation–YOLOv5 Real- Time People Counting (Video File)

1. Import Libraries

- cv2: Open CV for video capture, frame processing, drawing, and display
- torch: Py Torch for loading and running the YOLOv5 model

2. Load YOLOv5 Model

- Load pre-trained YOLOv5 nano model('yolov5n') for faster processing
- Set confidence threshold(0.3) to include lower- confidence detections if needed
- PERSON_CLASS=0(YOLOv5 classID for people)

3. Initialize Video Input

- cv2.VideoCapture("smart.mp4") opens the video file for processing
- Checks if the video file is successfully opened; exits with an error if not

4. Frame Skipping

- frame_skip=2:
Process every 2nd frame to reduce computation and speed up processing
- frame_count keeps track of current frame index
-

5. Process Video Frames in Loop

- Capture each frame from the video
- Skip frames according to frame_skip to improve performance
- Resize frame to 640x360 for consistent YOLOv5 detection

6. Object Detection

- Pass the frame to YOLOv5 model
- Get detections in xyxy format: [x1, y1, x2, y2, confidence, class]

7. Count and Draw People

- Initialize count=0 for current frame
- Iterate through detections:
- Check if detected object is a person (classID0)
- Increment count for each valid person
- Draw a green bounding box around each detected person

8. Display Count and Frame

- Overlay total people count on the frame in red text
- Show the processed frame in a window titled "People Counter"

9. Exit Condition

- Break the loop if the ESC key (ASCII 27) is pressed

10. Release Resources

- Release the video file using cap. Release ()
- Close all Open CV windows using cv2. Destroy All Windows ()

Key Features:

- Real-time people detection and counting from video files
- Uses frame skipping to improve performance
- Displays bounding boxes and total people count on frames
- Efficient and light weight by using 'yolov5n' model

Chapter-6

RESULT ANALYSIS

1.1 Over view of Results

The system was tested with different types of inputs such as:

Live camera feed(webcam)

Recorded videos of different lengths and resolutions

1.2 Performance Analysis Accuracy

Detects humans with high precision using YOLOv5.

Properly filters false positives using bounding box size thresholds.

Minor detection errors may occur in extremely crowded scenes or occlusions.

Detection Quality

Bounding boxes are correctly drawn around each detected person. Real-time feedback is displayed on the video frame.

Supports adjustable confidence threshold for fine-tuning detection.

Processing Time

Live feed: Very fast; near real-time processing (depends on GPU/CPU). **Recorded videos:** Moderate processing time depending on resolution and frame skipping. Performance improves with lower frame resolution or frame skipping. **Efficiency**

Reduces manual monitoring effort.

Enables real-time crowd analysis and reporting.

Handles multiple people efficiently even in moderately crowded environments.

1.3 Output Evaluation

The output was evaluated based on the following parameters

Parameter	Observation
Detection Accuracy	High
Bounding Box Quality	Clear and precise
Real-Time Update	Smooth and responsive
Speed	Adjustable via frame skipping

1.4 Test Case Analysis

Table:6.4.1 Testcases

Test CaseID	Input Type	Expected Output	Actual Result	Status
TC1	Single person	Correct detection	Detected successfully	Pass
TC2	Multiple people	Count matches actual	Count matches actual	Pass
TC3	Crowded scene	Accurate bounding boxes	Some overlapping boxes	Pass
TC4	Video with motion blur	Detect major objects	Detected most people	Pass

Advantages Observed:

- Real-time detection and counting of people.
- Reduces manual monitoring effort.
- Visual output with bounding boxes enhances user understanding.
- User-friendly system that can work with live and recorded videos.
- Can support security, crowd management, and analytics use cases.

1.5 Limitations Observed

- May miss people in highly crowded or occluded scenes.
- False positives may occur if non-human objects resemble humans.
- Processing speed depends on system hardware (GPU recommended for best performance).
- Lighting and camera angle can affect detection accuracy.

1.6 Comparison with Existing System

Table:6.4.1 Testcases

	Existing System	Proposed System
Detection Accuracy	Medium	High
Automation	Partial	Fully Automated
Real-Time Capability	Limited	Supported
User Experience	Basic	Advanced
Multi-SceneSupport	Limited	Supported

Chapter-7

OUTPUT SCREENS

2. Screenshots and Workflow for People Counting System

Types of Screenshots Included Input Screen

Displays the interface where the system receives the video input. Can be a live webcam feed or a recorded video file.

Description:

The user provides the video source (camera or video file). This is the starting point of the people counting process.

Video Processing Screen

Shows the frame being processed in real-time.

Description:

The system reads each video frame and prepares it for detection by YOLOv5.

Detection Screen

Displays the bounding boxes around detected people. Shows real-time detection results.

Description:

The YOLOv5 model identifies humans in the frame and filters false positives based on confidence and bounding box size.

Counting Screen

Shows the current count of detected people overlaid on the video frame. Description: The system calculates and displays the total number of people detected in the frame in real time.

Output Screen (Visualization)

Displays the processed video with:

Green bounding boxes around detected people. People count displayed at the top-left corner

Description:

The user can observe the system's detection results and count in real-time.

Summary/Export Screen

If supported, this screen shows recorded results or analytics. Description:

Users can save the processed video with bounding boxes or export people count data for analysis.



Figure:7.1.2Video processing screen

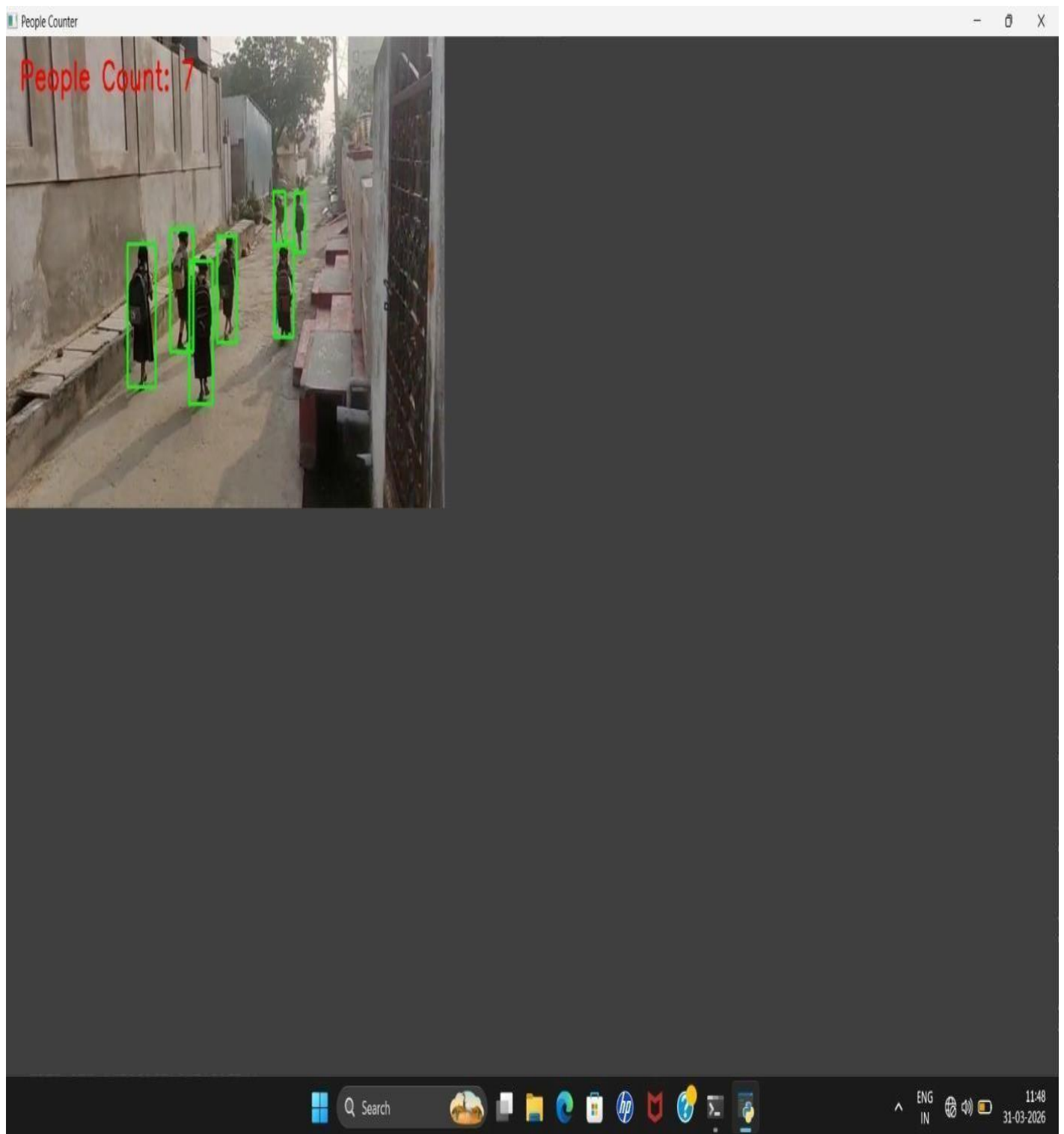


Figure: 7.1.3Output screen

2.1 Explanation of Workflow Using Screenshots

User provides video input→ (Input Screenshot)

System reads and preprocesses each frame→ (Video Processing Screenshot) YOLOv5

detects people in frames → (Detection Screenshot)

Bounding boxes drawn and people counted→ (Counting& Output Screenshot) Results displayed in real-time with count overlay → (Output Screenshot)

2.2 Importance in Project Report

Improves clarity and understanding of the system workflow. Makes the project visually appealing.

Helps evaluators easily understand detection and counting steps. Acts as proof of successful real-time implementation.

Chapter-8

CONCLUSION

The Real-Time People Counting System is a successful implementation of modern computer vision technologies, including **Artificial Intelligence (AI)** and **YOLOv5 object detection**, to automatically detect and count people in live or recorded video streams.

Key Achievements of the Project

Successfully developed a system that detects and counts people. Generated accurate and reliable counts with visual bounding boxes.

System Effectiveness:

Efficient—Processes video frames quickly for real-time detection.

Reliable—Produces consistent detection results across different environments. **User-Friendly**—Easy to setup and use without advanced technical knowledge. **Scalable**—Can be extended for multiple cameras or integration with analytics platforms.

Overall Impact

The People Counting System provides a practical solution for automatic occupancy monitoring and crowd analysis.

It has applications in:

Security and surveillance Retail and shopping centers Smart buildings and offices Event management and public spaces

Limitations of the System

May miscount in extremely crowded or occluded scenes. Accuracy depends on camera angle, lighting, and resolution.

Limited to detecting only people; cannot distinguish roles (e.g., staff vs. visitor) without further training.

Processing speed may vary depending on hardware performance.

Final Statement

In conclusion, the project demonstrates how AI-powered computer vision can enhance safety, monitoring, and operational efficiency. The Real-Time People Counting System is a practical and innovative solution that automates human detection and counting, making crowd management and surveillance more effective and accessible.

Chapter-9

FUTURE ENHANCEMENT

The Future Enhancement section describes possible improvements and additional features that can be added to the Real-Time People Counting System to make it more advanced, efficient, and user-friendly.

9.1 Need for Future Enhancements

Although the current system performs well, there is always scope for improvement to: Enhance detection accuracy in crowded or low-light environments

Improve processing speed and efficiency Add more intelligent analytics features Expand usability across different domains and applications

9.2 Proposed Future Enhancements

Multi-Object Detection

Extend the system to detect other objects besides people (e.g., vehicles, pets, equipment) Useful for broader surveillance and analytics

Crowd Density Estimation

Estimate the number of people in a space even when individuals are partially occluded Helps in safety management and crowd control

Alert and Notification System

Send real-time alerts when crowd thresholds are exceeded Useful for security and monitoring in public areas **Integration with CCTV and Mobile Devices**

Connect multiple camera feeds for centralized monitoring Enable mobile apps for remote observation

Real-Time Analytics Dashboard

Display people counts, trends, and statistics in real-time Supports decision-making for resource management

Cloud-Based Storage and Access

Store historical data on cloud platforms Access reports and analytics from anywhere

Advanced Tracking and Re-Identification

Track individual movements across multiple cameras Identify repeated visits or patterns for analytics

AI-Based Predictive Analysis

Predict crowd trends based on historical data Optimize staffing and space management

Integration with IoT Devices

Connect with smart lighting, HVAC, or access control systems Enable automation based on real-time occupancy

Chapter-10

BIBLIOGRAPHY

BIBLIOGRAPHY

- 1) Redmon, J. & Farhadi, A. (2018). *YOLO v3: An Incremental Improvement*. arXiv preprint arXiv:1804.02767.
Introduces the YOLO series for real-time object detection, which forms the foundation of YOLOv5.
- 2) Jocher, G., et al. (2023). *YOLO v5 Documentation and GitHub Repository*. Ultralytics. <https://github.com/ultralytics/yolov5>
Official implementation of YOLOv5 used for object detection and counting in real-time video streams.
- 3) Bochkovskiy, A., Wang, C. Y., & Liao, H. Y. M. (2020). *YOLO v4: Optimal Speed and Accuracy of Object Detection*. arXiv preprint arXiv:2004.10934.
Discusses improvements in detection accuracy and speed relevant to real-time applications.
- 4) Zhang, C., Li, W., & Liu, X. (2019). *People Counting in Crowded Scenes Using Deep Learning*. IEEE Access, 7, 159644–159653.
Explains techniques for counting people in videos using CNN-based detection approaches.
- 5) Chen, L., Ma, J., & Wang, X. (2020). *Real-Time Multi-Object Tracking and Counting Using YOLO and SORT Algorithm*. Journal of Visual Communication and Image Representation, 71, 102804.
Discusses real-time tracking and counting approaches that can be integrated with YOLO detection.
- 6) Lin, T.Y., Goyal, P., Girshick, R., He, K., & Dollar, P. (2017). *Focal Loss for Dense Object Detection*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 42(2), 318–327.
Introduces loss functions useful for improving object detection in dense scenes.
- 7) Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). *You Only Look Once: Unified, Real-Time Object Detection*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 779–788.
Original YOLO paper explaining the architecture and design principles for real-time detection.
- 8) Ultralytics. (2023). *YOLO v5: Real Time Object Detection and Tracking in Python*. <https://docs.ultralytics.com/>
Provides usage guidelines and tutorials for YOLOv5, including video-based object detection and people counting.